LAB 6

Code and Branch Coverage Analysis

Irrigation Scheduler

Student	Saiesh Shankar
USN	PES1UG24CS401
Repository	github.com/SaieshShankar/L6_L7_37_irrigation_scheduler
Application under test	irrigation_scheduler.py

Final result: 100% line coverage and 100% branch coverage

Prepared from the Lab 6 handout, the irrigation scheduler test files, and the terminal evidence supplied.

1. Objective and Overview

The objective of this lab is to understand the difference between code coverage and branch coverage, use coverage tools to measure test completeness, interpret coverage reports, and add test cases for untested paths and decision outcomes.

The application tested was an irrigation scheduler implemented in Python. Its core logic is organised into three functions:

should_irrigate() — decides whether a zone requires irrigation using soil moisture, rainfall, and plant growth stage.

water_volume() — calculates the water volume for a zone based on its area, growth stage, and temperature. It also rejects non-positive area values.

schedule() — iterates through irrigation zones, skips zones that do not require watering, stops when the remaining budget is insufficient, and records selected zones.

2. Tools and Environment

Component	Environment / tool
Language	Python 3.11.5
Test framework	pytest 8.3.4
Coverage tool	coverage.py 7.17.0 (as shown in terminal evidence)
Project	L6_L7_37_irrigation_scheduler / lab6

3. Coverage Concepts

Line (statement) coverage measures whether executable statements were run during testing. A statement is counted as covered when at least one test executes it.

Branch coverage measures whether the possible outcomes of decision points were exercised. For example, tests should exercise both true and false outcomes of relevant conditions, not merely execute the line containing the condition.

A high line-coverage percentage does not by itself guarantee that alternate decision outcomes have been tested. Branch coverage provides additional evidence about those paths.

4. Test Design and Execution

The initial test file contained two example tests. The first coverage run showed that much of the application logic had not been exercised. Additional tests were then added in two dedicated files:

- **CodeCoverageTest_PES1UG24CS401_application.py**
- **BranchCoverageTest_PES1UG24CS401_application.py**

The tests were designed to exercise the main functional cases and decision outcomes across the three application functions.

4.1 Test cases covered

Function	Test dimensions exercised
should_irrigate()	Moisture at and below thresholds; rainfall at relevant thresholds; seedling, flowering, and other growth stages; irrigation and no-irrigation outcomes.
water_volume()	Zero and negative area errors; seedling, flowering, and fallback stages; high, low, and normal temperature conditions.
schedule()	Empty zone list; skipping a zone; adding a zone to the plan; insufficient budget causing the loop to stop.

4.2 Commands used

```
python -m coverage erase
python -m coverage run -m pytest CodeCoverageTest_PES1UG24CS401_application.py
python -m coverage report -m
```

For branch coverage, the run was repeated with the `--branch` option and the branch-specific test file:

```
python -m coverage erase
python -m coverage run --branch -m pytest
BranchCoverageTest_PES1UG24CS401_application.py
python -m coverage report -m
```

The terminal evidence shows three tests passing in each dedicated test run.

5. Results and Analysis

The initial coverage run, using the original example test file, reported 29% line coverage and 24% branch coverage for the irrigation scheduler. After adding the dedicated test cases, both metrics reached 100%.

Metric	Initial result	Final result
Line coverage	29%	**100%**
Branch coverage	24%	**100%**
Tests in dedicated run	2 original tests	**3 passed per run**

5.1 Interpretation

The final line-coverage report records 38 statements, zero missed statements, and 100% coverage for irrigation_scheduler.py. The final branch-coverage report records 38 statements, 28 branches, zero partial branches, and 100% coverage for the application.

These results indicate that all measured executable statements and all measured branch outcomes in the application were exercised by the final tests. The test files themselves also report 100% coverage in the displayed runs.

6. Conclusion

The lab demonstrated the practical difference between line coverage and branch coverage. The initial tests executed only a limited portion of the irrigation scheduler. After adding targeted tests for normal cases, boundary values, error conditions, and alternate decision outcomes, the application achieved 100% line coverage and 100% branch coverage in the recorded runs.

The two dedicated test files were committed separately and pushed to the student's fork of the assigned GitHub repository. Terminal screenshots are included in the evidence section.

Appendix A. Initial Coverage Evidence

The first two terminal panels in the supplied evidence show the baseline run: 29% line coverage and 24% branch coverage for irrigation_scheduler.py.

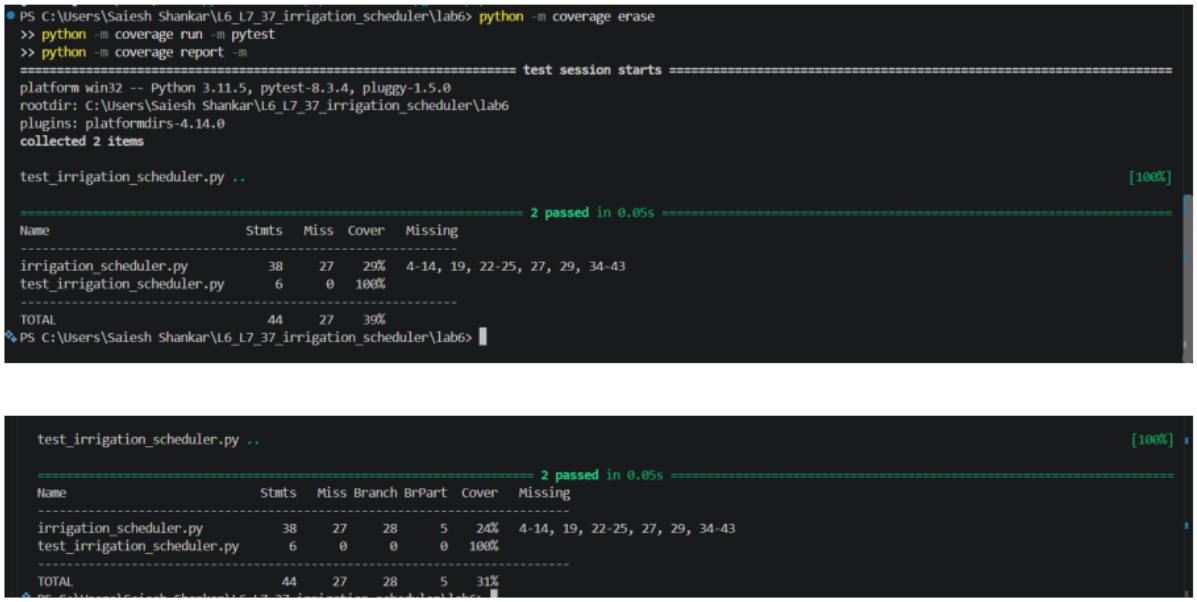


Figure A1. Initial coverage reports from the original test file.

Appendix B. Final Line Coverage Evidence

The supplied terminal screenshot shows the dedicated code-coverage test file running three tests successfully. The application report records 38 statements, zero missed statements, and 100% coverage.

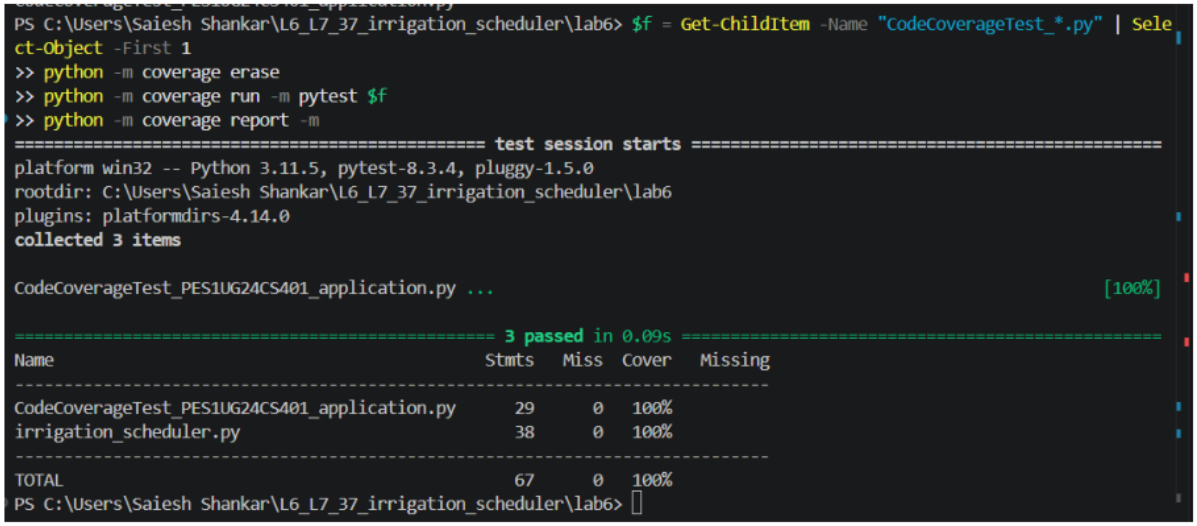


Figure B1. Final line-coverage run.

Appendix C. Final Branch Coverage Evidence

The supplied terminal screenshot shows the dedicated branch-coverage test file running three tests successfully. The application report records 28 branches, zero partial branches, and 100% coverage.

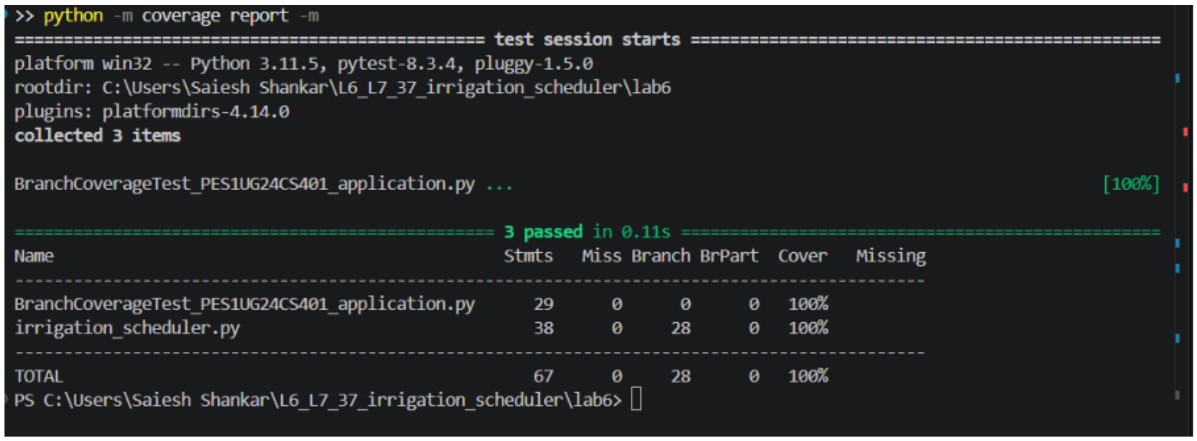


Figure C1. Final branch-coverage run.